Apollo-inspired EDU_ROBO Environmental Monitoring & Local-AI Ground Station

Baseline: 2026-10-11 · **Target schedule:** 1–2 weeks · **Hardware authority:** EDU_ROBO IoT Trainer Kit, ESP32-WROOM-32 DevKit V1 (30-pin), V14 / Lab Manual V4 reference sheet.

1. Project in one paragraph

Build a robust ESP32-based sensor-monitoring system whose firmware is organized around a mandatory custom application-level executive running on ESP-IDF/FreeRTOS. The executive schedules sensor jobs, manages fixed-capacity data structures, validates a numeric verb–noun command protocol, records faults and timing metrics, and continues operating when sensors or communications fail. A Fedora laptop acts as the ground station: one Python connection manager handles USB serial and Wi-Fi/MQTT, writes validated timestamped measurements to InfluxDB, exposes Grafana dashboards and structured logs, and routes board-originated commands to open the requested local interface. Open WebUI provides chat over a custom Python data/tool API; local inference is optional and only enabled after the installed Cactus-Compute/Needle and general-purpose model runtimes have been verified. The AI must retrieve measurements through deterministic tools and must never invent sensor data or block firmware operation.

2. Fixed decisions and scope

- **Timebox:** 7–14 days. Embedded executive, bounded structures, sensor acquisition, and reliability experiments are mandatory. Ground-station and AI features are staged; do not sacrifice the core to install every optional service.
- **Firmware:** ESP-IDF + FreeRTOS, C, target `esp32` (ESP32-WROOM-32 / classic ESP32).
- **Board:** EDU_ROBO V14, 30-pin ESP32-WROOM-32 DevKit V1. Treat the supplied V14 sheet as the pin-map authority, but verify physical wiring and resolve contradictions before enabling affected drivers.
- **Data:** InfluxDB for timestamped measurements and numeric system metrics; structured JSONL logs initially. Grafana for dashboards. Loki is optional, not required for the first build.
- **Connectivity:** USB serial and Wi-Fi/MQTT through one logical device/connection manager. Each transport has its own framing adapter but shares a versioned message envelope and command registry.
- **Commands:** Board-originated verb–noun commands can request the laptop to open Grafana, logs, ground-station overview, or Open WebUI. Connection-triggered opening is separately configurable. Launch only allowlisted URLs; never execute arbitrary shell commands from device messages.
- **AI:** Open WebUI + custom Python telemetry/tool service. Needle/Cactus-Compute is a candidate for constrained intent/tool routing only after API, model, latency, and compatibility tests. A separate compatible local general-purpose model may answer questions. AI is optional and never part of the real-time embedded path.
- **No cloud dependency:** Core sensing, executive, command validation, and USB diagnostics must work offline. Local AI and dashboard services may be unavailable without stopping firmware.

3. Actual hardware inventory and constraints

Confirmed installed/connected for this build

- ESP32-WROOM-32 DevKit V1, 30-pin.
- DHT11 module on GPIO 19.
- HC-SR04 module on ULTRA connector: trigger GPIO 25, echo GPIO 26 (verify the actual module output voltage before connection).
- HW-870 IR/Gas module on IR-GAS1 / CD4051 X0.
- Soldered LDR on GPIO 35 / ADC1.
- Soldered Hall-effect IC2 on CD4051 X1.
- Soldered TSOP1838 IR receiver on GPIO 1; active-low/decoded protocol behavior must be verified.
- Soldered MOC7811 slot/opto sensor on GPIO 34 (input-only; no internal pull-up).
- Soldered sounder/buzzer on GPIO 15, LEDs, CD4051 mux, and other board support circuitry.

Listed by the reference sheet but verify physical presence before enabling

LM35 on GPIO 39; sound sensor on GPIO 32; touch sensor on GPIO 18; vibration sensor on GPIO 23; ADXL335 axes on mux channels (the connector/channel mapping in the sheet is inconsistent); moisture sensor on X2; water-level/rain sensor on X3; line trackers on X4–X6; Sharp IR on a documented but conflicting channel; BMP180 on GPIO 25/26; GPS/GSM on GPIO 16/17; servo on GPIO 33. The 10 kΩ potentiometer is explicitly absent. DC motors, relay, 16×2 LCD, USB cable, and several mechanical/power parts are listed as absent. Do not assume they exist.

Pin map / electrical rules

- GPIO 19: DHT11.
- GPIO 25: HC-SR04 trigger; conflicts with BMP180 SDA.
- GPIO 26: HC-SR04 echo; conflicts with BMP180 SCL.
- GPIO 35: LDR, ADC1 input-only.
- GPIO 36: CD4051 common output, ADC1 input-only.
- GPIO 39: LM35 input if installed.
- GPIO 32: sound analog input if installed.
- GPIO 18: touch input if installed.
- GPIO 23: vibration input per direct-sensor table, but the reference contains a conflicting mention of ultrasonic echo; prefer the dedicated HC-SR04 mapping GPIO 25/26 and verify the PCB.
- GPIO 1: TSOP1838 and UART0 TX pin conflict. Boot/log output may interfere with IR reception; test before assigning physical IR command entry.
- CD4051 select lines: A=GPIO 4, B=GPIO 2, C=GPIO 5; output GPIO 36; INH tied to GND. Truth table: CBA 000=X0, 001=X1, 010=X2, 011=X3, 100=X4, 101=X5, 110=X6, 111=X7.
- The spec states GPIO signals must stay between 0 and 3.3 V and no pin should be treated as a power output. It gives 10 mA maximum per pin; keep GPIO drive conservative.
- HC-SR04 modules commonly have 5 V echo on some variants. Measure/check the actual module and use a divider/level shifter if the echo can exceed 3.3 V. Do not connect a 5 V signal directly to ESP32 GPIO.
- The sheet says the micro-USB data cable is not included. Confirm a data-capable cable and USB-UART enumeration before starting serial work.
- Do not drive missing motors/relay/LCD or attach optional modules until power, connector, and pin-sharing details are verified.

4. Sensor support policy

Implement a sensor registry so the executive and telemetry schema can support the entire verified inventory. A registry entry contains stable sensor ID/type, driver, pin/channel, enabled state, sample interval, priority class, units/calibration metadata, last status, timeout/error counters, and validity. Support is not a claim that hardware is present. Emit values only for detected/configured sensors. Unknown calibration means publish raw ADC/relative values, not invented physical units.

Initial active sensors

1. `dht11`: temperature °C and relative humidity % on GPIO 19.
2. `ldr`: raw ADC and relative light index on GPIO 35; calibrated lux only after calibration.
3. `hw870_x0`: raw ADC/voltage on CD4051 X0; do not claim gas concentration in ppm without sensor identity, calibration, and warm-up requirements.
4. `hall_x1`: raw/threshold event on X1 if the onboard sensor's output and channel are verified.
5. `hcsr04`: distance on GPIO 25/26 only after echo-voltage safety is verified.
6. `moc7811`: pulse/event input on GPIO 34 once output circuitry and pull-up behavior are checked.
7. `tsop1838`: optional IR event receiver only after resolving GPIO 1/UART0 conflict; not required for first milestone.

The CD4051 is one shared analog output, not eight simultaneous ADC inputs. One mux manager serializes selection, settling, ADC conversion, and channel tagging. The spec's channel assignments are alternatives where noted; never configure multiple sensors as occupying the same channel at once. Optional channels: X2 moisture/ADXL335 Z; X3 rain/water-level; X4–X6 line trackers; X7 potentiometer (absent on this build). Enable only verified modules.

5. Embedded architecture

Layering

1. **Hardware drivers:** GPIO, ADC1, DHT11, HC-SR04, mux, optional digital event inputs.

2. **Sensor registry:** configuration, sample validity, units, calibration, health counters.

3. **Custom executive (mandatory):** application-level job lifecycle, bounded ready queue, fixed job table, event queue, sensor history ring buffer, dispatch policy, timing/deadline metrics, and fault handling. FreeRTOS remains the underlying kernel; the custom executive is not a replacement kernel.

4. **Command service:** numeric verb–noun parser, parameter validation, preconditions, idempotency and response codes.

5. **Transport adapters:** USB serial and MQTT; shared envelope and logical device identity.

6. **Diagnostics:** structured events, counters, runtime status, test hooks.

No blocking sensor calls, network operations, database writes, browser launches, or inference should run inside a scheduler-critical section. Use bounded timeouts and queue messages between tasks. Queue-full conditions must be counted and handled explicitly; never silently overwrite unread data unless a documented drop-oldest policy applies to a specific ring buffer.

Initial scheduling policy

- Priority class 0: critical command reception, executive tick/health, urgent fault capture.
- Priority class 1: sensor acquisition and mux transactions.
- Priority class 2: telemetry formatting/transmission and noncritical diagnostics.

These are project policy classes, not hard real-time guarantees. Measure dispatch latency and deadlines rather than claiming determinism without evidence.

Bounded data structures

- Fixed-size job table with explicit states and slot reclamation.
- Fixed-capacity ready queue with stable priority/tie-breaking.
- Fixed-capacity event queue.
- Fixed-size sensor history ring buffer.
- Fixed-size command and response buffers; reject oversize inputs.
- Bounded telemetry batch and retry queue.
- No unbounded allocation in periodic acquisition paths. If dynamic allocation is used during initialization, handle allocation failure.

6. Command protocol

Commands use a versioned, length-delimited/line-framed envelope. Conceptual human form: `VERB NOUN [key=value ...]`. Numeric codes are project-defined and are not historical Apollo AGC codes. Registry entries define parameters, allowed state, timeout, local effect, and response. USB serial uses newline-delimited JSON or a length-prefixed frame; MQTT carries the same JSON payload on topics scoped to the device. Pick one wire encoding before implementation and share golden test vectors between C and Python.

Initial command registry (v1)

Verb code / name	Noun code / name	Behavior	Class
00 NOP	00 NONE	Parser/protocol test	Read-only
01 INITIALIZE	01 SYSTEM	Initialize application services	State change
01 INITIALIZE	02 SENSORS	Initialize verified sensor drivers	State change
01 INITIALIZE	03 TRANSPORTS	Initialize available transports	State change
02 START	01 SYSTEM	Start monitoring	State change
02 START	02 SENSORS	Start enabled acquisitions	State change
02 START	03 TELEMETRY	Enable transmission	State change
02 START	04 SELF_TEST	Run bounded self-test	Diagnostic
03 STOP	02 SENSORS	Stop acquisition cleanly	State change
03 STOP	03 TELEMETRY	Stop telemetry transmission	State change
04 RESET	01 FAULTS	Clear eligible latched faults	State change
04 RESET	02 STATISTICS	Reset diagnostic counters	State change
05 SET	01 PRIORITY	Change only allowed job priorities	State change
05 SET	02 SAMPLE_INTERVAL	Set validated sensor interval	State change
05 SET	03 THRESHOLD	Set configured threshold	State change
06 GET	01 STATUS	Overall status	Read-only
06 GET	02 CONFIGURATION	Active configuration	Read-only
06 GET	03 SENSOR_LIST	Present/enabled sensor registry	Read-only
06 GET	04 TIME	Monotonic uptime/time quality	Read-only
16 DISPLAY	01 STATUS	Current device status	Read-only
16 DISPLAY	02 EXECUTIVE	Executive state	Read-only
16 DISPLAY	03 QUEUES	Queue/job-table state	Read-only
16 DISPLAY	04 FAULTS	Active faults	Read-only
16 DISPLAY	65 ENVIRONMENT	Latest DHT11 readings	Read-only
16 DISPLAY	66 LIGHT	Latest LDR reading	Read-only
16 DISPLAY	67 DISTANCE	Latest HC-SR04 reading	Read-only
16 DISPLAY	68 ANALOG	Enabled analog channels	Read-only
16 DISPLAY	69 MOTION	Motion data if installed	Read-only
16 DISPLAY	70 EVENTS	Recent events	Read-only
17 HISTORY	01 EVENTS	Recent event history	Read-only
17 HISTORY	02 FAULTS	Fault history	Read-only
17 HISTORY	03 EXECUTIVE	Scheduler history	Read-only
17 HISTORY	65 ENVIRONMENT	Environmental history	Read-only
17 HISTORY	66 SENSORS	History for `sensor_id`	Read-only
24 DIAGNOSTICS	01 SYSTEM	System health	Read-only
24 DIAGNOSTICS	02 SENSORS	Sensor errors/freshness	Read-only
24 DIAGNOSTICS	03 QUEUES	Queue depth/overflow	Read-only
24 DIAGNOSTICS	04 TIMING	Latency/deadline metrics	Read-only
24 DIAGNOSTICS	05 MEMORY	Heap/stack high-water marks	Read-only
24 DIAGNOSTICS	06 TRANSPORTS	USB/Wi-Fi/MQTT health	Read-only
30 OPEN	01 DASHBOARD	Laptop opens Grafana dashboard	Laptop action
30 OPEN	02 LOGS	Laptop opens log view	Laptop action
30 OPEN	03 AI	Laptop opens Open WebUI	Laptop action
30 OPEN	04 GROUND	Laptop opens ground-station overview	Laptop action
31 ASK	01 SYSTEM	Open AI with system-health context	Laptop action
31 ASK	65 ENVIRONMENT	Open AI with environmental context	Laptop action

Verb code / name	Noun code / name	Behavior	Class
31 ASK	66 SENSOR	Open AI with selected sensor context	Laptop action
32 EXPORT	01 TELEMETRY	Request bounded telemetry export	Read-only job
32 EXPORT	02 DIAGNOSTICS	Generate diagnostic report	Read-only job
33 RUN	01 SELF_TEST	Approved bounded self-test	Diagnostic
33 RUN	02 SENSOR_TEST	Test selected available sensors	Diagnostic
33 RUN	03 STRESS_TEST	Controlled test build only	Test-only

A command is not implemented merely because it appears in the registry. Mark each entry `planned`, `implemented`, or `verified` in the source registry. `STOP SYSTEM`, arbitrary shell, arbitrary URLs, unrestricted DB queries, and unauthenticated remote actuator commands are intentionally excluded. Sensor selection uses a validated `sensor_id` parameter instead of creating a unique verb/noun for every sensor.

Response contract

Every command response includes protocol version, device ID, session ID, command ID, verb/noun, status (`accepted`, `completed`, `rejected`, `failed`, `unsupported`), reason/error code, and timestamps where available. Long jobs return `accepted` plus a job ID, followed by a completion/failure event. Deduplicate repeated command IDs across USB and MQTT. Reject unknown codes, invalid parameters, oversized messages, stale sessions, and unauthorized requests.

Physical command entry

The board has a soldered TSOP1838 IR receiver on GPIO 1, but GPIO 1 is also UART0 TX and can conflict with serial logging. An IR remote is a possible later physical input after testing and resolving that conflict. The kit has no keypad or LCD installed. Therefore, first implement and test the command dispatcher via USB serial and MQTT; then choose IR remote or a small keypad for laptop-independent physical entry. Do not claim a board-only physical command path exists until tested.

7. Laptop ground station

- **Connection manager:** one logical device record across USB and MQTT; track device ID, firmware/protocol version, session, last heartbeat, transport state and message sequence. Deduplicate commands/telemetry where appropriate.
- **Serial adapter:** newline-delimited or length-prefixed framing, bounded frame size, explicit port selection, reconnect handling.
- **MQTT adapter:** per-device topic namespace, QoS chosen per message class, authenticated broker; commands must be authenticated and constrained.
- **Ingestion:** validate schema, units, timestamp, sensor ID and sequence; batch writes to InfluxDB without blocking serial/MQTT receive loops.
- **Application launcher:** map `30 OPEN` events to allowlisted local URLs; open one dashboard on connection only if configured; cooldown and duplicate suppression prevent repeated tabs. No shell interpolation from received data.
- **Structured logs:** JSONL locally at first, with event timestamp, severity, device, event type, command/session IDs and bounded context. Loki optional.
- **Offline behavior:** firmware keeps sampling and records bounded local history/faults when disconnected; reconnect reports gaps and sequence continuity. RAM buffer does not survive power loss; persistent spool is a later optional feature.

8. InfluxDB and Grafana

Use InfluxDB as the primary laptop-side time-series store. Suggested measurements: `environment`, `light`, `analog_sensor`, `distance`, `pressure`, `motion`, `sensor_status`, `executive_metrics`, and `device_status`. Emit only fields supported by present, verified hardware. Tags should be low-cardinality (`device_id`, `sensor_id`, `sensor_type`, optional configured location); keep changing readings, error counts, and sequence numbers in fields rather than tags.

Store acquisition time and laptop receipt time separately when useful. Preserve raw ADC values until calibration is verified. Distinguish invalid/stale/missing data from a valid zero. Dashboard panels: current sensor values and freshness; time-series trends; sensor error/freshness; device connectivity; queue depth/overflow; job latency/deadline misses; transport reconnects; recent fault events. Grafana data source uses least-privilege credentials. Never expose database credentials to the model.

9. Local AI and Open WebUI

Open WebUI is the chat front end. A custom Python service owns typed read-only tools and validated InfluxDB/log queries. Candidate tools: `get_latest_readings`, `query_measurements`, `get_sensor_status`, `get_recent_events`, `get_executive_metrics`, `compare_time_windows`, and `summarize_sensor_health`. Apply allowlists, query-range caps, result limits, unit metadata and data-quality flags. No arbitrary SQL/Flux and no direct actuator control in the initial AI tool set.

Needle/Cactus-Compute must be verified on the actual Fedora laptop: package/runtime version, callable API, model availability, tool-call behavior, CPU/GPU support, latency and memory. Use it for constrained intent/tool routing only if tests support that role. Select a separate general-purpose local LLM for explanations if compatible. Laptop has an i5-10300H, GTX 1650 Ti Mobile, 16 GB RAM and 8 GB currently reported in use; this is a constraint, not a promise that a particular model fits. Start with small quantized models and sequential model loading; do not keep multiple models resident by default. Deterministic statistics remain the fallback if AI is unavailable.

AI answer contract: report units and time windows; identify data gaps; distinguish observed values from interpretation; never invent readings; include supporting timestamps/events; say when evidence is insufficient. Inference runs only on the laptop and cannot block the ESP32 executive.

10. Repository layout

```
apollo-edu-robo/
  README.md
  docs/PROJECT_BLUEPRINT.md
  firmware/
    CMakeLists.txt
    main/
      app_main.c
      executive/
      data_structures/
      sensors/{direct,adc,mux,optional}/
      command/
      transports/
      diagnostics/
  ground_station/
    connection_manager/
    serial_transport/
    mqtt_transport/
    command_router/
    application_launcher/
    telemetry_ingestion/
    api/
    logging/
  ai/{model_adapters,needle_tools,telemetry_tools,query_validation,prompts}/
  grafana/{dashboards,provisioning}/
  tests/{unit,integration,stress,hardware}/
  results/{raw,processed,figures}/
```

11. One-to-two-week build plan

Time	Deliverable	Exit condition
Days 1–2	Fedora ESP-IDF setup, baseline build/flash/monitor; DHT11 and LDR proof	Reproducible build, serial logs, stable basic acquisition
Days 3–4	Custom executive, fixed job table, ready/event queues, history ring, command registry	Unit tests for bounds, queue full, state transitions and slot reuse
Days 5–6	Mux manager, HW-870 X0, Hall X1, HC-SR04 after electrical check; serial protocol	Verified sensor IDs/units, channel correctness, malformed-frame rejection
Days 7–8	Reliability tests and metrics	Results for queue saturation, timeouts, invalid data, deadlines and recovery
Days 9–10	Python connection manager, MQTT, InfluxDB, Grafana, structured logs, launcher	USB and MQTT feed the same logical device; command opens correct local view once
Days 11–12	Open WebUI, Python tools, Needle/model verification	At least one evidence-grounded query works or AI limitation is documented
Days 13–14	Integration, documentation, repeatability	USB-only, Wi-Fi-only and dual-link tests; report includes raw results and limitations

For a seven-day version, protect the embedded executive and reliability experiments first; defer AI and optional sensors rather than claiming untested completion.

12. Required reliability experiments

1. **Scheduling/load:** dispatch latency, execution time, priority order and deadline misses.

2. **Capacity exhaustion:** fill queues/job table; verify explicit rejection/overflow counters and safe recovery.

3. **Sensor failure:** unplug/timeout/invalid value; unrelated jobs continue and stale data is flagged.

4. **Mux correctness:** verify all configured channels, settling, identity and cross-channel contamination; only test physically wired channels.

5. **Transport failure:** USB disconnect, Wi-Fi loss, MQTT reconnect, duplicate suppression and sequence gaps.

6. **Command safety:** valid, invalid, oversized, duplicate, unsupported and unauthorized commands; only allowlisted URLs open.

7. **Resource stability:** queue wraparound, job reclamation, heap/stack high-water marks and repeated operation.

8. **AI evidence quality:** known test dataset, expected values, timestamps/units, gaps and insufficient-evidence responses.

Record test setup, firmware commit, input, expected result, actual result, raw logs, measurements, and a pass/fail conclusion. Do not fabricate results.

13. Definition of done

- Firmware builds reproducibly for `esp32` and runs on the physical board.
- Custom executive and all required bounded data structures are implemented and tested.
- Every physically present sensor in scope is identified and supported, or a documented electrical/pin conflict blocks activation.
- Serial command protocol is versioned, bounded and validated; command registry tracks planned/implemented/verified status.
- USB serial and Wi-Fi/MQTT share one logical connection manager.
- InfluxDB stores timestamped measurements; Grafana shows readings, freshness and executive health.
- Board-originated `OPEN` commands launch only the corresponding allowlisted local view; connection startup is optional and deduplicated.
- Open WebUI uses the custom Python tool service; local models are used only after compatibility verification.
- Reliability experiments have real recorded evidence, limitations and reproducible instructions.

14. First development milestone

1. Confirm a data-capable Micro-USB cable is available.

2. Install/activate ESP-IDF on Fedora using Espressif's official Linux setup guide.

3. Connect the board and identify its `/dev/ttyUSB*` or `/dev/ttyACM*` device. Capture `dmesg`/`journalctl` output and `ls -l /dev/serial/by-id/` if available.

4. Build and flash a minimal ESP-IDF application that prints firmware version, reset reason, uptime and a heartbeat.

5. Only after serial is stable, add one sensor driver at a time. Verify HC-SR04 echo voltage before measuring distance; confirm mux X0 and X1 physical behavior before interpreting the HW-870/Hall values.

Official references: ESP-IDF Linux installation, ESP-IDF build/flash/monitor workflow, Grafana InfluxDB data source, Open WebUI documentation.